

AI for Fight The Landlord

Lei Qian, Zihao Zhang, Zixiu Lin, Jingqi Xu

School of Information Science and Technology
ShanghaiTech University

{qianl, zhangzh2, linzx, xujq}2024@shanghaitech.edu.cn

Abstract

Fight the Landlord is a challenging imperfect-information card game with a large combinatorial action space, hidden hands, and strong role-dependent cooperation and competition. In this project, we study several reinforcement learning and search-based approaches for building agents for this game.

We first set the baseline: random and greedy agents. The random agent samples uniformly from the legal actions, while the greedy agent uses a small set of hand-crafted rules based on card combinations.

We then explore reinforcement learning methods, including policy gradient and q-learning, to learn policies that can adapt to different game states and roles.

Finally, we try adversarial search techniques, such as Monte Carlo Tree Search (MCTS), to improve decision-making by simulating future game states and evaluating potential actions.

Our experiments and implementation process show ;

Introduction

Fight the Landlord, also known as Dou Dizhu, is a three-player card game with one landlord playing against two farmers. Although the rules are easy to describe, the decision problem is difficult for an artificial agent. A player must choose among many legal card combinations, reason about the hidden hands of the other two players, and adapt its strategy according to whether it is the landlord, the landlord's upper farmer, or the landlord's lower farmer. The game therefore combines a large action space, imperfect information, and team-based competition.

From a reinforcement learning perspective, the main difficulty is that the true game state is not fully observable. A player only knows its own hand, the public action history, the last played move, the number of remaining cards for each player, and its role. The exact cards held by the other players are hidden. As a result, a naive Markov decision process over complete hands is not directly usable at decision time, while a state representation based only on the current player's hand loses too much information about the actual game context.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.



Figure 1: Visualization of Fight the Landlord game board

State Space and Imperfect Information

Motivation

Methods

Random

The random agent is the simplest baseline in our experiments. At each decision point, the environment provides the agent with an information set I , including the current player's hand, the last move, public history features, and the legal action set $\mathcal{A}(I)$. The random agent ignores all strategic information and samples uniformly from the legal actions:

$$a \sim \text{Uniform}(\mathcal{A}(I)).$$

In implementation, this corresponds directly to `random.choice(infoset.legal_actions)`. Since the action is always sampled from the environment-generated legal action list, the agent never needs to check card-type validity by itself. This baseline is useful because it gives a lower-bound reference for whether a learned or heuristic policy is making meaningful decisions beyond simply playing legal cards.

Greedy

The greedy baseline (we use it as a baseline for comparison the effect of our methods) corresponds to the `RLCardAgent` used in our evaluation code. Unlike the random agent, it uses a small set of hand-crafted rules based on card combinations. The agent first converts the environment’s rank-coded cards into a compact string representation, where ranks are ordered as

3, 4, 5, 6, 7, 8, 9, *T*, *J*, *Q*, *K*, *A*, 2, *B*, *R*.

It then distinguishes between two cases: leading a new round and following a previous move.

When the agent is leading, it decomposes its hand into several structural combinations using `combine.cards`. The decomposition extracts combinations in a fixed priority order: rocket, bomb, trio, trio chain, solo chain, pair chain, pair, and finally solo cards.

After this decomposition, the agent searches for a combination containing the minimum card in its current hand and plays that combination. This produces a simple greedy leading policy: it tends to release low cards while preserving larger cards and special combinations for later.

When the agent is following another player’s move, it uses `RLCard`’s card-type table to identify the type and rank of the last move. Among all legal non-pass actions with the same card type, it chooses the one with the smallest rank that can beat the previous move. Formally, if $\mathcal{B}(I)$ is the set of legal responses with the same type as the last move, the selected action is

$$a^* = \arg \min_{a \in \mathcal{B}(I)} \text{rank}(a).$$

If no such response exists, the agent may pass. In particular, when the previous valid move was made by the other farmer and the current agent is also a farmer, it passes to avoid beating its teammate. If none of the above rules applies, the agent falls back to choosing a random legal action.

This greedy policy is still very limited: it does not perform long-horizon planning, does not estimate hidden hands, and does not learn from experience. However, it is stronger than a random baseline because it follows basic Dou Dizhu principles such as playing the lowest sufficient response, preserving high cards, and avoiding unnecessary interference with a teammate.

Probability

The probability inference agent is motivated by the imperfect-information problem, and it uses a cheaper one-step response model instead of explicitly sampling many complete hidden states. The key question is: after we play an action a , how likely is the next player to beat it, pass, or spend a strong response such as a bomb? This idea is implemented by `probability.inference.py` and used by `probabilistic.response.agent.py`.

Given an information set I , the agent first constructs the remaining unknown card multiset $U(I)$ by removing its own hand and all public played cards from the full deck. If the next player has m cards left, then a possible hidden hand

is treated as a size- m draw from $U(I)$. For a candidate response r , let $R(r)$ be the multiset of cards required to play that response. The probability that the responder can physically hold this response is estimated by a multivariate hypergeometric count:

$$p_{\text{has}}(r) = \frac{\#\{h : |h| = m, R(r) \subseteq h, h \subseteq U(I)\}}{\binom{|U(I)|}{m}}.$$

In the code, this is computed by `contains_probability`. It uses dynamic programming over card ranks rather than sampling hidden hands, so the estimate is deterministic and relatively fast. Candidate responses are generated by `possible_response_actions`: it enumerates legal symbolic actions that could beat the target action, including same-type higher responses, bombs, and rocket.

The holding probability alone is not enough, because a player may choose not to beat even when they can. Therefore, `estimate_response_distribution` combines card availability with a simple behavioral model. Each candidate response receives an unnormalized weight

$$w(r) = p_{\text{has}}(r) \exp\left(\frac{s(r, I)}{\tau}\right),$$

where $s(r, I)$ is a heuristic attractiveness score and τ is a temperature parameter. The score prefers cheap minimal responses, gives a large bonus when the response can finish the responder’s hand, encourages enemies to beat dangerous actions, and discourages teammates from beating their own side unless they can finish. Bombs and rocket are penalized unless the situation is urgent.

Pass is also modeled explicitly. If p_{beat} is the approximate probability that the responder has at least one beating response, then the raw pass mass is

$$w(\text{pass}) = (1 - p_{\text{beat}}) + p_{\text{beat}} \rho_{\text{pass}},$$

where ρ_{pass} is a role-dependent strategic pass parameter. For example, enemies have a lower strategic pass rate, while teammates have a higher one because they often should not block the current player’s move. The final response distribution is normalized as

$$P(r \mid I, a) = \frac{w(r)}{w(\text{pass}) + \sum_{r' \in \mathcal{R}(I, a)} w(r')}.$$

The agent then evaluates each root action by combining an immediate action score with the expected value of the next player’s response:

$$Q(a) = S_{\text{imm}}(a, I) + \lambda \sum_{r \in \mathcal{R}(I, a) \cup \{\text{pass}\}} P(r \mid I, a) V_{\text{resp}}(a, r, I).$$

Here S_{imm} rewards finishing, reducing hand badness, playing useful structures such as chains or trios, and releasing low cards, while penalizing unnecessary high cards,

bombs, and bad passes. The response value V_{resp} is negative when an enemy is likely to beat the action, especially if the enemy can finish, but positive when the enemy passes and the agent keeps initiative. For teammate responses, the model gives a large bonus if the teammate can finish and otherwise discourages unnecessary interference.

To avoid making the policy worse than a strong simple baseline, the probability agent uses the greedy RLCARD policy as an anchor. It first obtains the greedy action, then scores a pruned set of candidate root actions using the probability model. The probability-based action is allowed to replace the greedy action only in controlled cases, such as immediate finishing, clearly better leading structures, or stopping a dangerous enemy. This makes the method a fast probabilistic one-step lookahead policy rather than a full POMDP solver: it does not search many future rounds, but it directly models the most important hidden variable for the next decision, namely whether the next player can and will respond.

Adversarial Search

The adversarial search agent is designed for decision-time planning under imperfect information. Unlike value-based agents, it does not learn parameters from self-play. Instead, it asks the following local question at every move: given the current information set, which legal action is most robust against likely responses from the other two players?

The main difficulty is that Fight the Landlord is not a perfect-information game. The current player observes only an information set

$$I = (h_i, H, m_{\text{last}}, c, r_i),$$

where h_i is the player's own hand, H is the public action history, m_{last} is the last valid move, c records the remaining card counts of the three players, and r_i is the role of the current player. The hidden hands of the other two players are unknown. Therefore, a direct minimax search over the true state is impossible.

To approximate the hidden state, we use determinization. Let $\Omega(I)$ denote the set of all complete states consistent with the information set. The agent samples N possible complete states $s_1, \dots, s_N \sim \Omega(I)$ by randomly assigning the unknown cards to the two unobserved hands according to their remaining card counts. For a root action a , the sampled search value is

$$Q_{\text{search}}(I, a) = b(I, a) + \frac{1}{N} \sum_{k=1}^N V_d(T(s_k, a)),$$

where $b(I, a)$ is an immediate tactical bonus or penalty, T is the state transition after playing a , and V_d is a depth-limited search value. The term $b(I, a)$ handles short-term tactical cases such as finishing the hand, blocking a dangerous enemy, releasing a teammate, avoiding unnecessary bombs, and penalizing risky high singles.

Inside each determinized state, the search is role-aware. If the current node belongs to the root agent, the agent assumes it can choose the best continuation:

$$V(s) = \max_{a \in A_s} V(T(s, a)).$$

For teammate nodes, using a pure maximum is too optimistic because the teammate does not know the root player's hidden cards and may not follow the searched line perfectly. We therefore combine a policy expectation with a small maximum component:

$$V_{\text{team}}(s) = (1 - \mu) \sum_{a \in A_s} \pi_{\tau}(a | s) V(T(s, a)) + \mu \max_{a \in A_s} V(T(s, a)).$$

Here π_{τ} is a softmax policy over hand-crafted action scores, and μ controls how much we trust the teammate to find the best cooperative line.

For enemy nodes, a pure minimax update would be too pessimistic. It would assume that the opponent knows the determinized full state and always selects the most damaging action. However, using only a soft policy can also be too optimistic, because it may underweight the opponent's strongest punishments. We therefore use a symmetric mixed model: most of the value comes from a bounded-rational policy expectation, while a small component still accounts for the worst adversarial response:

$$V_{\text{enemy}}(s) = (1 - \nu) \sum_{a \in A_s} \pi_{\tau}^{\text{enemy}}(a | s) V(T(s, a)) + \nu \min_{a \in A_s} V(T(s, a)).$$

The action probabilities are computed from heuristic action scores:

$$\pi_{\tau}(a | s) = \frac{\exp(g(s, a)/\tau)}{\sum_{a'} \exp(g(s, a')/\tau)}.$$

Here ν controls how much pessimism is retained in the opponent model. This model is less paranoid than minimax but still biased toward strong responses. In practice, the search also prunes actions to a small top- K candidate set at both the root and internal nodes, which keeps the computation tractable.

When the search reaches a terminal state, the value is a large positive or negative constant depending on whether the root team wins:

$$V(s_{\text{terminal}}) = \begin{cases} +R_{\text{win}}, & \text{if the root team wins,} \\ -R_{\text{win}}, & \text{otherwise.} \end{cases}$$

At non-terminal leaf nodes, the agent falls back to a hand-crafted state evaluation. This evaluation rewards the root team for having fewer and better structured remaining cards, penalizes dangerous enemies with very few cards left, and adds bonuses for initiative, last valid control, bombs, and high control cards.

Finally, the agent does not blindly trust the sampled search result. Because determinization and shallow search

can be noisy, the chosen action must be sufficiently better than the baseline greedy candidate or the second-best candidate:

$$a_{\text{search}} = \arg \max_a Q_{\text{search}}(I, a).$$

The search action is accepted only if

$$\begin{aligned} & Q(a_{\text{search}}) - Q(a_{\text{base}}) \geq \delta_{\text{base}}, \\ \text{or } & Q(a_{\text{search}}) - Q(a_{\text{second}}) \geq \delta_{\text{second}}. \end{aligned}$$

Otherwise, the agent falls back to the baseline action. This gate makes the agent more stable: adversarial search is used when it gives strong evidence for an improvement, but the policy avoids being dominated by sampling noise or incorrect hidden-card assumptions.

Overall, this method can be viewed as a practical approximation to imperfect-information minimax: hidden cards are sampled, a shallow team-aware search is performed on each sampled complete state, and a protected decision rule combines the search result with a strong heuristic baseline.

Results